1. Introduction to FastAPI

What is FastAPI?
FastAPI is a **modern, high-performance web framework** for building APIs with Python 3.6+. It is built on top of **Starlette** (for the web parts) and **Pydantic** (for data validation).

- It is designed to be **fast** — performance on par with Node.js and Go.
- It automatically generates **OpenAPI** (formerly Swagger) documentation.
- It is **type-hinted** (Python type annotations), which enables excellent editor support, auto-completion, and automatic data validation.
- Ideal role: Acts as the **bridge** between your frontend (e.g., Streamlit, React, mobile apps) and your data layer (databases, AI models like OpenAI, business logic).

Key Advantage: You write simple Python functions, and FastAPI turns them into production-ready API endpoints with minimal boilerplate.

---

2. Why Choose FastAPI?

Core Benefits (with explanations):

- **High Performance**:
  FastAPI is one of the fastest Python web frameworks. It uses ASGI (Asynchronous Server Gateway Interface) and can handle thousands of concurrent requests efficiently. Benchmarks often show it matching or exceeding Node.js (Express) and Go (Gin/Fiber).

- **Rapid Development**:
  Increases productivity by 200–300% compared to older frameworks like Flask or Django.
  Reasons: Automatic validation, serialization, documentation, dependency injection, and great IDE support.

- **Fewer Bugs**:
  Strong typing + Pydantic validation catches errors at development time or request time before they reach your business logic. Reduces runtime surprises.

- **Intuitive & Beginner-Friendly**:
  Clean syntax, excellent error messages, and interactive docs make it accessible for students while remaining powerful for professionals.

When to use FastAPI?

- AI/ML backends (chatbots, RAG systems, agents)
- Microservices

- Mobile/web app backends
- Any project needing clean REST or WebSocket APIs

---

3. How an API Works (Restaurant Metaphor)

Think of an API like a restaurant:

Component	Metaphor	Real-World Meaning	Technology in this Course
Customer	You (user)	Frontend (Streamlit app)	Streamlit (port 8501)
Waiter	FastAPI Backend	Receives request, processes it, returns response	FastAPI (port 8000)
Kitchen	Data/AI Layer	Database, OpenAI API, business logic	PostgreSQL, OpenAI, etc.

Request-Response Flow:

1. Frontend sends HTTP request (GET/POST/etc.) to FastAPI.
2. FastAPI validates input, runs the function, interacts with database/AI.
3. FastAPI returns JSON response.
4. Frontend displays the result.

This **separation of concerns** (backend vs frontend) makes applications more maintainable, scalable, and testable.

---

4. Your First Endpoint

Basic Structure:

```
from fastapi import FastAPI

app = FastAPI(title="AI Chatbot API", version="1.0.0")

@app.get("/")
def home():
    return {"message": "Welcome to the AI Chatbot API!"}
```

How to Run:

```
uvicorn main:app --reload    # main.py is the file
```

- Visit `http://127.0.0.1:8000` → see JSON response.
- Visit `http://127.0.0.1:8000/docs` → interactive Swagger UI.

Decorators:

- `@app.get("/path")` → HTTP GET
- `@app.post("/path")` → HTTP POST
- `@app.put()`, `@app.delete()`, `@app.patch()` etc.

---

5. The Four Core Operations (CRUD)

Operation	HTTP Method	Purpose	Example in Chatbot App
Create	POST	Add new resource	Start a new chat session
Read	GET	Retrieve resource(s)	Load chat history or a message
Update	PUT/PATCH	Modify existing resource	Edit a message or metadata
Delete	DELETE	Remove resource	Delete a chat session

Best Practice: Follow RESTful conventions:

- `GET /chats` → list all chats
- `GET /chats/{chat_id}` → get one chat
- `POST /chats` → create new chat
- `DELETE /chats/{chat_id}` → delete chat

---

6. Dynamic URLs with Path Parameters

Example:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/chats/{chat_id}")
def get_chat(chat_id: str):
    # chat_id will contain whatever is in the URL, e.g., /chats/f7a2b9c1
    return {"current_chat": chat_id, "status": "retrieved"}
```

- Path parameters are **type-converted** automatically (int, float, UUID, etc.).
- You can add validation and metadata with `Path()` from `fastapi`.

```
from fastapi import Path

@app.get("/chats/{chat_id}")
def get_chat(chat_id: str = Path(..., min_length=8, max_length=36)):
    ...
```

---

7. Validating Data with Pydantic

Pydantic is the heart of FastAPI's data handling.

Example Request Model:

```
from pydantic import BaseModel
from typing import Optional

class MessageRequest(BaseModel):
    message: str
    user_id: Optional[str] = None
    temperature: float = 0.7   # default value

@app.post("/send")
def send_message(req: MessageRequest):
    # req is already validated and parsed
    return {
        "received_text": req.message,
        "temperature_used": req.temperature
    }
```

What happens automatically:

- Type checking (`str`, `int`, `float`, `list`, `dict`, custom models)
- Required field enforcement
- Automatic 422 Unprocessable Entity responses for bad data
- Conversion (string → int, JSON → Python objects)
- Rich error messages

Advanced Pydantic Features (recommended for students):

- Field constraints: `Field(gt=0, le=100)`

- Validators (`@validator` , `@root_validator`)
- Nested models
- Enums, Literal types

---

8. Free Interactive Docs (Swagger UI + ReDoc)

FastAPI auto-generates beautiful documentation at:

- `/docs` → **Swagger UI** (interactive – try endpoints directly in browser)
- `/redoc` → ReDoc (cleaner, more readable)

Benefits:

- No need to write separate documentation.
- Test APIs before building frontend.
- Shows request/response schemas automatically.

You can customize with OpenAPI metadata:

```
app = FastAPI(
    title="AI Chatbot API",
    description="Backend for intelligent conversation system",
    version="1.0.0",
    contact={
        "name": "Your Name",
        "email": "you@example.com"
    }
)
```

---

9. Full-Stack Integration (FastAPI + Streamlit)

Architecture:

- **FastAPI** (port 8000): Business logic, database, AI calls (OpenAI, LangChain, etc.)
- **Streamlit** (port 8501): User interface (buttons, chat input, display)
- Communication: `requests` library (or `httpx` for async)

Typical Flow in Streamlit:

```
import requests
```

```
response = requests.post(
    "http://localhost:8000/send",
    json={"message": user_input}
)
result = response.json()
```

Advantages of Separation:

- Frontend can be replaced (React, Vue, mobile) without touching backend.
- Backend can be scaled independently.
- Easier testing and team collaboration.